365 Account Management — Setup Guide

SharePoint lists & the Power Automate flow needed to run the web part.

Build the four lists by hand (no PnP in this tenant), then wire up one Power Automate flow for the O365 path. This guide is the source of truth.

CONTENTS

How it works

The web part branches on the **Group ID format** of each managed group:

Group ID	Path	What happens
GUID (O365 / Entra group)	Request + flow	The web part writes a `Pending` row to the *Group Membership Requests* list. A **Power Automate flow** (elevated identity) re-checks authorization, performs the Microsoft Graph change, and writes the result back. The web part polls.
Integer (SharePoint site group)	Direct REST	The web part changes membership directly via SharePoint REST as the signed-in user — **no flow**. Bounded by that user's SharePoint permissions.

You need **4 SharePoint lists** and **1 Power Automate flow** (for the O365 path only). List titles are configurable in the web part property pane; defaults are shown. All four lists live on one site (the "List site URL" in the property pane).

5 rules before you start

> **1. Internal column names must match EXACTLY and have NO spaces.** SharePoint freezes a column's *internal* name from its display name at creation and encodes spaces as `_x0020_`. Create each column with the exact space-free name shown, *then* rename the display label if you want it prettier.

- **2. Every listed column must exist** — the app `$select`s them by name; a missing one fails the whole query. Exceptions are marked optional.
- **3. Person & Lookup columns are used by their `…Id` field.** Create `RequestedBy`; the app uses `RequestedById`. Don't create `RequestedById` literally.
- **4. Don't re-create built-ins** built-in: `Title`, `Created By` (Author), `Modified`, `ID`.
- **5. List titles** must match the defaults below, or be set per page in the property pane → *Data source (lists)*.

Legend — **Filled by:** Admin = you populate by hand · App = the web part writes it · Flow = Power Automate writes it back · System = SharePoint maintains it.

List 1 — `Managed Groups`

Property: `groupListTitle` · One row per managed group/office · **Admin-populated, read-only to the app.** All columns required except `GroupType`.

Column (internal name)	Type	Notes
`Title` built-in	Single line of text	Group display name.
`GroupId` required	Single line of text	Entra/O365 group **GUID** (prod) or SharePoint site-group **integer** (dev). The app can't act on a row without it.
`Description`	Multiple lines of text	Shown in the UI.
`Mail`	Single line of text	Group email.
`MailNickname`	Single line of text	Alias / title fallback.
`Visibility`	Single line of text	Public / Private (free text).
`CreatedDateTime`	Date and Time	Group creation date (separate from built-in *Created*).
`IsTeamsConnected`	Yes/No	Teams-connected flag.
`SiteUrl`	Hyperlink	Optional same-tenant **site-collection root** for SharePoint-group actions.
`SiteTitle`	Single line of text	Friendly site/group title; fallback.
`GroupType` optional	Single line of text	Optional to create — the app retries without it. Flags dynamic / distribution groups.

List 2 — `Group Membership Requests`

Property: `requestListTitle` · The queue **and** audit log for O365 changes · App writes a `Pending` row, the flow writes the result back. All columns required except `RequestedBy`.

Column (internal name)	Type	Filled by	Notes
`Title` built-in	Single line of text	App	e.g. "Add Member: Jane Doe".
`Action`	Choice	App	Exact values: **Add Member**, **Remove Member**.
`GroupId`	Single line of text	App	Target O365 group **GUID**.
`GroupName`	Single line of text	App	Group display name.
`MemberId`	Single line of text	App	Member's Entra object id (GUID) — the flow uses this for the Graph call.
`MemberDisplayName`	Single line of text	App	
`Status`	Choice	App + Flow	Exact values: **Pending**, **Completed**, **Failed**. Default = `Pending`.
`ResultMessage`	Multiple lines of text	Flow	Outcome / error text.
`RequestedOn`	Date and Time	App	ISO submit timestamp.
`TargetUserPrincipalName`	Single line of text	App	Member UPN.
`TargetUserEmail`	Single line of text	App	Member email.
`CorrelationId`	Single line of text	App	GUID for app↔flow correlation.
`OfficeGroupRecord`	Lookup → `Managed Groups` (show `Title`)	App	Links the request to its group row. App writes `OfficeGroupRecordId`.
`Justification`	Multiple lines of text	App	**Must exist** — not dropped on failure. Backs the "Require a reason" toggle.
`MemberEntraId`	Person or Group	App	Target member as a site user (REST `MemberEntraIdId`). **Must exist.**
`RequestedBy` optional	Person or Group	App	Initiator (REST `RequestedById`). App retries without it if absent.
`AuthorizationChecked`	Yes/No	Flow	App reads.
`AuthorizationResult`	Multiple lines of text	Flow	App reads.
`Created By` (`Author`) built-in	Person	System	The flow authorizes on this. App filters recent requests by it.
`Modified` built-in	Date and Time	System	Shows when the flow last updated the row.

Permissions: break inheritance so requesters get *Add-items only + read-own*, and only the flow identity can edit `Status` / `Authorization*`. Enable version history (the audit log). Index `Status` for the flow's Pending sweep.

List 3 — `Group Management Authorized Admins`

Property: `authorizedAdminsListTitle` · Who may manage which groups · **Admin-populated, read-only to the app.**

One row per admin. `OfficeGroupRecord` is a **multiple-value** lookup — in a single row, select *all* the groups that admin manages. (The app also still reads the older one-row-per-group layout, so you can migrate gradually.)

Column (internal name)	Type	Notes
`User` required	Person or Group (single)	The authorized admin. The app & flow filter on this (REST `UserId`) — **index it.**
`OfficeGroupRecord` required	Lookup → `Managed Groups` (show `Title`), **Allow multiple values** ✓	All groups this admin may manage — pick several in the one row. The app reads each and pulls `Title` / `GroupId` / `SiteTitle`.

Switching an existing list to multi-select: SharePoint won't let you toggle "Allow multiple values" on a lookup that already has data. **Delete** the existing `OfficeGroupRecord` column and **re-create it with the same name** and *Allow multiple values* checked, then consolidate to one row per admin.

⚠️ **This changes the flow too.** The flow's server-side authorization re-check reads this list. It must test whether the requested group is **in** the requester's multi-value collection (see the flow section). Change the list column and the flow's check **together**, or O365 requests will fail authorization.

List 4 — `Group Site Permissions` optional list

Property: `sitePermissionsListTitle` · Feeds the "Used on these sites" panel · **Admin-populated, read-only to the app.** The whole list is optional; create all four columns to make the panel work. One row per (group → site).

Column (internal name)	Type	Notes
`GroupId`	Single line of text	The group's **GUID** (matches `Managed Groups.GroupId`). App filters on it — **index it.** Plain text, not a lookup.
`SiteName`	Single line of text	Panel label; results sorted by this — **index it.**
`SiteUrl`	Hyperlink	Clickable site link.
`Permission`	Single line of text	e.g. Full Control / Edit / Read (free text).

The Power Automate flow (O365 requests only)

SharePoint site-group changes are applied directly by the web part and never reach the flow. The flow exists purely for the **O365 (GUID) group** path.

Trigger & identity

- **Trigger:** *When an item is created* on `Group Membership Requests`. **Concurrency = 1** (serialize).
- **Runs as:** a service account that is **Owner of every managed O365 group**, or an app registration with application permission `GroupMember.ReadWrite.All`. (The web part itself holds only read scopes — this identity is the elevation.)

Fields the flow reads / writes

Reads (from the created item)	Writes back (terminal)
`Action`, `GroupId` (target group), `MemberId` (member object id), `Author` / `AuthorId` (the requester — the **only** trusted identity), `CorrelationId`, `Justification`, member display fields.	`Status` = **Completed** / **Failed** (exact casing), `ResultMessage`, `AuthorizationChecked`, `AuthorizationResult`.

> **Never authorize on client-writable fields** (`RequestedBy` / `RequestedById`, `OfficeGroupRecordId`). Only the server-stamped `Author` / `AuthorId` is trustworthy.

⭐ Authorization re-check — multi-value (the current logic)

4a. Resolve the requester's SharePoint user id from the created item:

```
GET {listsSite}/_api/web/lists/getbytitle('Group Membership Requests')/items({ID})?$select=AuthorId
```

4b. Read that admin's authorized groups, expanding the multi-value lookup:

```
GET {listsSite}/_api/web/lists/getbytitle('Group Management Authorized Admins')/items
    ?$select=Id,OfficeGroupRecord/GroupId
    &$expand=OfficeGroupRecord
    &$filter=UserId eq {AuthorId}
Header:  Accept: application/json;odata=nometadata
```

> Filter by `UserId` (the single-value `User` column — filterable, indexed), *not* the group. SharePoint can't reliably `$filter` a multi-value lookup's sub-field, so fetch the requester's row(s) and test the group membership **inside the flow**.

4c. Build the set of **all** `OfficeGroupRecord[].GroupId` across the returned row(s). Authorize **iff** the request's `GroupId` is in that set (case-insensitive GUID compare). Under `odata=nometadata`, `OfficeGroupRecord` is a plain **array** per row (`[ {GroupId}, {GroupId}, … ]`); under `odata=verbose` it is `{ "results": [ … ] }`.

- **Authorized** → continue; set `AuthorizationChecked = Yes`, `AuthorizationResult = "Authorized"`.
- **Not authorized** (group not in the set, or no rows) → **fail closed:** `Status = Failed`, `AuthorizationChecked = Yes`, a clear `AuthorizationResult` / `ResultMessage`, and **make no Graph call**.

Perform the change (only if authorized)

Using `GroupId` (target) and `MemberId` (member object id):

```
Add Member:
POST https://graph.microsoft.com/v1.0/groups/{GroupId}/members/$ref
Body: { "@odata.id": "https://graph.microsoft.com/v1.0/directoryObjects/{MemberId}" }
  → treat HTTP 400 "already exists" as success (idempotent)

Remove Member:
DELETE https://graph.microsoft.com/v1.0/groups/{GroupId}/members/{MemberId}/$ref
  → treat HTTP 404 "not found" as success (already not a member)
```

On success → `Status = Completed`. On any other Graph error → `Status = Failed` with the error in `ResultMessage`.

Invariants the web part relies on

1. **Authorize on `AuthorId` only** (server-stamped); ignore client-writable fields for security.
2. **Fail closed** — any error / missing row / ambiguity → `Status = Failed`, no Graph call.
3. **Exact `Status` casing** — `Completed` / `Failed` (never `Complete`, `Success`, …). The web part polls for these literally; a mis-cased value hangs the UI until timeout.
4. **No row left `Pending`** — every branch writes a terminal status. A scheduled "sweep" flow that fails stale `Pending` rows is a recommended backstop.
5. **Idempotent** — tolerate already-member / not-member.

Final checklist

- ☐ All four lists created on one site; titles match the property-pane *Data source (lists)* values (and the *List site URL*).
- ☐ Every column uses the exact internal names above (no spaces); Person/Lookup columns created by base name.
- ☐ `Group Management Authorized Admins` → `OfficeGroupRecord` is **multi-value**; one row per admin.
- ☐ Indexes: `User` (List 3), `GroupId` + `SiteName` (List 4), `Status` (List 2).
- ☐ Request list hardened: requesters Add-items + read-own; only the flow edits `Status` / `Authorization*`; version history on.
- ☐ Power Automate flow: trigger on the request list; **multi-value authorization re-check**; fail-closed; exact `Status` casing.
- ☐ Flow identity is Owner of every managed O365 group (or app-registration `GroupMember.ReadWrite.All`).
- ☐ Tenant admin approved the web part's Graph scopes: `GroupMember.Read.All`, `User.ReadBasic.All`, `User.Read.All`, `ProfilePhoto.Read.All` (add `Group.Read.All` if the group search 403s).
